

One Developer Is All You Need: A Case Study of an AI-Augmented One-Person Squad in a Brownfield Enterprise

Marcelo Vilas Boas*, Gustavo Pinto†, Edward Roberto Monteiro*, Vinicius Fernandes Caridá* and Danilo Ribeiro‡

*Itaú Unibanco, São Paulo, Brazil

Email: {marcelo.boas, edward.monteiro, vinicius.carida}@itau-unibanco.com.br

†Universidade Federal do Pará, Belém, Brazil

Email: gpinto@ufpa.br

‡CESAR School, Recife, Brazil

Email: dmr@cesar.school

Abstract—AI agents are enabling single engineers to absorb work previously distributed across cross-functional squads, but there is almost no structured evidence of how to design or evaluate this configuration in a regulated enterprise setting. Without such evidence, organizations adopting the one-person squad model lack guidance on which design decisions make it viable and which conditions cause it to break down, leaving practitioners to rely on industry anecdotes and inflated productivity claims. We report a single-case study in which one staff engineer, supported by four AI agents under a Spec-Driven Development workflow, delivered a brownfield product initiative scoped for a four-person squad in half the planned time, with 90% AI-generated code acceptance, full integration test coverage, and an above-85% reduction in direct staffing cost—outcomes driven not by AI capability but by specification quality and the engineer’s institutional knowledge. This inverts the prevailing reading of AI coding tools as productivity boosters for less experienced developers: in the one-person squad, the senior engineer’s expertise is exactly what makes the gains possible, which means scaling this model depends on growing T-shaped engineers, not on acquiring more AI tooling.

Index Terms—AI agents, one-person squad, brownfield software engineering, enterprise software development, cognitive load, team compression, AI-augmented development

I. INTRODUCTION

How software development teams should be organized is one of the oldest questions in software engineering. Brooks [1] established that adding engineers to a project increases communication overhead faster than it adds capacity. Agile methods operationalized this insight into a durable norm: small, cross-functional teams of roughly three to nine people, coordinated through short iterations and shared ownership. This configuration dominated industrial practice for two decades because it balanced coordination cost against specialist coverage in a way that manual labor alone could not circumvent. Solo developers existed, but were largely confined to freelance work, open-source side projects, or early-stage startups; in regulated or legacy-intensive enterprise settings, single-person delivery was considered structurally infeasible.

Generative AI has started to destabilize that assumption. GitHub Copilot, introduced in 2021, was among the first tools to bring code generation into professional workflows; a controlled experiment by Peng et al. [2] found that developers with access to Copilot completed a standardized task 55.8% faster than a control group. Later work complicated the picture: Becker et al. [3] found that experienced open-source developers using AI tools on mature repositories took 19% *longer* to complete tasks, showing that productivity gains depend heavily on context. More recently, the field has moved from AI-as-assistant to AI-as-agent: multi-agent systems now assign specialized agents to distinct software engineering roles—requirements, design, implementation, review, testing—and coordinate their outputs autonomously [4]. This shift changes what a small team can accomplish, because the limiting factor is no longer how many humans are available but how well those humans can direct and evaluate machine-generated work.

One visible consequence of this shift is the re-emergence of minimal team configurations. Industry observers report that AI-augmented engineers can now cover the full delivery lifecycle—requirements, architecture, implementation, testing, compliance—work that previously required cross-functional squads [5], [6]. The limiting case is the *one-person squad*: a single engineer who, supported by configured AI agents, assumes end-to-end responsibility for a product initiative. The concept has gained rapid traction in industry discourse, but it has not been studied rigorously: reported multipliers vary widely, the design decisions that make it work are poorly documented, and evidence from regulated enterprise environments—where compliance, brownfield systems, and institutional knowledge raise the stakes—is almost entirely absent.

This paper addresses that gap through a single-case practitioner-researcher study at a large Brazilian financial institution. One staff engineer, working under a Spec-Driven Development (SDD) workflow [7] with four AI agents—StackSpot for discovery and requirements [8], Devin for spec-

ification drafting and non-core implementation, and GitHub Copilot for core development—delivered a product initiative originally scoped for a four-person squad over six sprints. We keep the brownfield character of the system in view throughout: active legacy integrations, regulatory constraints, and implicit organizational knowledge are conditions the model had to operate against, not variables we tried to neutralize. The study describes how the one-person squad was set up, examines what it actually shipped, and isolates the conditions under which the arrangement held. Findings are summarized below.

- **On outcomes:** the squad delivered five features in three sprints against a six-sprint plan, achieving a 50% reduction in time-to-market relative to the team’s historical baseline; 90% of AI-generated code was accepted without structural modification, all integration tests passed at sprint close, and only one post-validation defect was found.
- **On conditions:** specification quality was the single most important determinant of AI output quality, particularly in the brownfield context where undocumented legacy contracts were the most frequent source of rework; the core/non-core partition proved a repeatable tool-assignment heuristic; and the model functioned as a multiplier of existing expertise rather than a substitute for it—its viability depended on the engineer’s institutional knowledge serving as the quality gate that the removed team members would otherwise have provided.

II. BACKGROUND

A. Specification-Driven Development

Specification-Driven Development (SDD) is an emerging paradigm for AI-assisted software construction in which natural-language specifications—rather than code—are treated as the primary engineering artifact [7]. The premise is that LLM-based coding assistants such as GitHub Copilot and Devin have shifted the locus of developer effort away from writing implementation details and toward declaring intent: requirements, interface contracts, pre- and post-conditions, and acceptance criteria. Under SDD, the developer’s role is to produce a specification precise enough that an AI agent can generate, test, and refine the corresponding code with minimal ambiguity, while the specification itself remains the durable artifact that captures design decisions and supports later maintenance.

SDD has clear lineage in earlier methodologies, most notably Test-Driven Development (TDD), which similarly inverts the traditional code-first workflow by requiring tests to be written before the implementation. Recent work has shown that combining TDD-style loops with LLMs improves the accuracy and reliability of generated code, by giving the model an executable acceptance criterion to converge against [7]. SDD generalizes this idea one step upstream: tests themselves become an output of the specification rather than a separate authoring step, and the specification governs both implementation and verification. For our case, this paradigm is particularly

Listing 1. Canonical SDD specification template used as a structured prompt. Concrete content is omitted; placeholders in angle brackets denote the instantiation points.

```
## TASK
<single, unambiguous statement of the work to be done>
<scope boundary: what is in, what is out>
<list of files or modules expected to be created or modified>

## TDD AND VALIDATION
<acceptance tests that MUST pass; the agent treats this as the stopping criterion and the contract for completion>

Unit tests:
1. <assertion>
2. <assertion>

Integration tests:
1. <assertion>

Quality gates enforced in CI:
- Coverage threshold: <value>
- Static analysis: <tool, severity>

## SUBMIT WORK
- Branch naming convention: <pattern>
- Commit message convention: <pattern>
- Pull request target branch: <branch>
- PR title and description requirements: <pattern>
- Required reviewers or approval rules: <list>

## FORBIDDEN ACTIONS
1. <directories or files the agent must not touch>
2. <classes of changes that require human escalation>
3. <dependency, schema, or contract changes out of scope>
4. <security or compliance constraints, e.g. secrets handling>
```

relevant because it makes specification quality—not model capability—the binding constraint on output quality, a claim we revisit empirically in Section V. Listing 1 presents our SDD prompt template.

B. One-Person Squad

Individual developers working without teammates are a recognized configuration in software engineering. Brooks [1] formulated the foundational tradeoff: team size carries a coordination cost that grows non-linearly with headcount, so reducing team size eliminates overhead but also removes the specialist coverage that larger teams justify. In conventional settings, the loss of specialist coverage and peer review has limited solo development to small, low-stakes projects.

AI agents alter this tradeoff. He, Treude, and Lo [4], in their survey of LLM-based multi-agent systems for software engineering, map applications across the full software development lifecycle and characterize multi-agent systems as a way to manage real-world project complexity through specialization and coordination among agents. Under this paradigm, a single human engineer directing configured AI agents is not a traditional solo developer bearing the full cognitive weight of every role, but an orchestrator whose non-human teammates absorb routine specialist work. The one-person squad investigated in this paper instantiates exactly this configuration in a brownfield enterprise setting: one experienced engineer, multiple AI agents assigned to distinct functional roles, operating under a

structured workflow that reserves human judgment for high-stakes decisions.

III. OUR ONE-PERSON SQUAD

Our one-person squad was structured around four AI agents covering the full delivery lifecycle: a **product manager agent** for discovery and requirements, a **specification agent** for refinement, and two **developer agents** (core and non-core modules) for implementation and testing. After these stages, a human-driven validation step closed each feature at homologation. Table I summarizes the configuration. Each agent was pre-loaded with domain-specific knowledge, organizational standards, and project constraints before work began, and the overall workflow followed SDD [7]: detailed feature specifications—covering architectural decisions, interface contracts, acceptance criteria, and compliance constraints—served as structured prompts for subsequent AI-driven implementation, separating the design phase from execution.

TABLE I
AGENT ROLES IN THE ONE-PERSON SQUAD

Stage	Tool	Mode
Discovery & requirements	StackSpot agent	Human-in-the-loop
Specification	Devin	Human-in-the-loop
Development (core)	GitHub Copilot	Human-in-the-loop
Development (non-core)	Devin	Autonomous

These agents were configured as follows:

- **Product Manager Agent.** The product manager role was implemented as a custom agent built on StackSpot [8], Itaú’s internal AI agent platform. The agent was configured with a project-specific knowledge base covering the digital signature domain and the integration contract with LACUNA, the external signature provider. Knowledge bases for both the provider and the internal product area were ingested upfront so that the agent could carry the necessary business context into feature decomposition and user story generation.
- **Specification Agent.** Specification and its refinement was performed by Devin, which has native multi-repository access. During each refinement cycle, all 9 repositories belonging to the project were selected as context, and Devin produced the specifications used in the subsequent coding phase based on the requirements generated in the discovery step. This multi-repo grounding was essential in the brownfield setting, since most features touched contracts spread across more than one service.
- **Developer Agents.** The development phase was governed by a dual-module strategy that allocated supervision by domain judgment: the more judgment a task required, the more human oversight it received. **Core modules**—business rules, domain use cases, and user-facing logic—

were developed with GitHub Copilot in agent mode under a human-in-the-loop workflow, with the engineer reviewing each generation against the specifications produced in the previous step. **Non-core modules**—infrastructure, API integrations, message queue configuration, and other boilerplate code—were delegated to Devin operating autonomously, again driven by the same specifications. Both tracks operated under explicit guardrails enforced in the CI/CD pipeline: accessibility checks (WCAG 2.1 AA), security scans, and test coverage targets of 90% had to be satisfied before any artifact could progress. Unit and integration tests were generated by the same agents, inferring expected behaviors, boundary conditions, and failure scenarios from documented requirements.

The final homologation stage remained human-driven by design. Once a deployment to the homologation environment passed all automated guardrails, a human reviewer performed last-mile validation—performance testing, UI verification, and acceptance checks—before approving the production deployment.

IV. RESEARCH METHODOLOGY

This study employs a single-case study following the methodological framework proposed by Yin [9] and the guidelines for conducting and reporting case study research in software engineering established by Runeson and Höst [10]. Runeson and Höst synthesized practices from software engineering research and the broader case study literature to define what a rigorous case study report must address. Their checklist specifies four main elements: the case and its context (§ IV-A); the research questions (§ IV-B); the data collection procedures (§ IV-C); and the data analysis procedures (§ IV-D). We structure this section around these elements.

A. Case Context

The study was conducted at Itaú Unibanco, one of the largest private banking institutions in Latin America, headquartered in São Paulo, Brazil. The institution operates under the regulatory frameworks of the Central Bank of Brazil (BACEN) and the Brazilian Securities and Exchange Commission (CVM), and manages a digital product portfolio that serves over 100 million clients across retail, corporate, and investment banking segments.

The project analyzed in this study involved the development of a digital signature platform tailored for non-account holders. The system has been running in production since October 2025, built on an architecture composed of four microservices and a single user-facing interface. The project scope encompassed five distinct features, each decomposed into five user stories, resulting in a total of 25 user stories.

The development phase took place between August 2025 and September 2025, organized into three sprints of three weeks each. Technical execution was led by an engineer with over 8 years of professional experience, including a 4-year tenure within the institution’s specific technology stack and business domain.

The brownfield nature of the system is an essential element of this case study. Unlike greenfield settings, brownfield projects impose legacy integration constraints, implicit architectural knowledge, compliance requirements, and dependency management overhead that stress-test AI-augmented workflows in ways that controlled experiments cannot reproduce.

B. Research Questions

The study is organized around two research questions:

- RQ1:** What delivery and quality outcomes did the one-person squad achieve on this project?
- RQ2:** Under what conditions does the one-person squad model succeed or break down?

RQ1 is descriptive: it asks what the configuration actually produced when measured against the same team’s historical baseline on comparable scope, covering throughput, time-to-market, test coverage, and post-release defects. **RQ2** is analytical: it asks which design decisions, contextual factors, and engineer characteristics determined whether the model held up over the nine-week period, and which conditions caused it to require human escalation. Together, the two questions move from *what happened* to *why it happened that way*, which is the structure recommended by Runeson and Höst [10] for case studies aiming to inform transferability rather than statistical generalization.

C. Data Collection

Data was collected through practitioner-researcher participation over a nine-week period spanning three development sprints. The primary source was Itaú Unibanco’s internal delivery platform—comparable in function to Jira but integrated end-to-end with the bank’s engineering toolchain—which records BCP, lead time, test coverage, and pipeline outcomes as features advance through the workflow. The engineer-researcher had read access to the platform and extracted records directly from it. Two categories of metrics were captured.

- **Delivery metrics.** We collected the following metrics: features and user stories completed per sprint, total time-to-market against the original six-sprint plan, and the squad’s historical throughput on prior projects of comparable scope within the same product domain. Throughput was normalized using CI&T’s Business Complexity Points (BCP) framework [11], an internal complexity-scoring instrument adopted as the institution’s standard productivity measure across squads. The metrics are grouped into three categories—scope delivered, flow and time, and throughput—to separate what was produced from how quickly it moved through the pipeline.
- **Quality metrics.** We collected the following metrics: unit test coverage, integration test pass rate (passing cases over total defined cases across all features), accessibility compliance against WCAG 2.1 AA [12], and defect count from post-validation and acceptance testing. Coverage was measured by JaCoCo (on Kotlin and Java backend

services) and by Jest (on the Angular frontend); end-to-end frontend testing was performed with Cypress. Accessibility was validated in two layers: an *automated check* embedded in the pipeline and a *manual review* by a dedicated accessibility specialist who signed off on each feature before release.

D. Data Analysis

Data analysis followed a within-case analysis strategy [9]. Delivery and quality metrics were computed directly from project artifacts—build and test pipeline reports. The data was subsequently reviewed by the squad’s engineering managers as part of the institution’s standard sprint reporting cycle, providing a layer of validation independent from the engineer-researcher. No statistical inference was applied; the analysis is descriptive, given the single-case design.

V. RESULTS

In this section, we present the results organized around the two research questions.

A. RQ1: What delivery and quality outcomes did the one-person squad achieve on this project?

The one-person squad delivered five features (25 user stories) in three three-week sprints.

1) *Delivery metrics:* Table II reports the delivery metrics collected over the three sprints, grouped into scope delivered, flow and time, and throughput.

TABLE II
DELIVERY METRICS COLLECTED DURING THE STUDY

Metric	Sprint 1	Sprint 2	Sprint 3	Total
<i>Scope delivered</i>				
Features completed	0	2	3	5
User stories completed	7	9	9	25
BCP delivered	79	377	434	890
<i>Flow and time</i>				
Sprint duration (weeks)	3	3	3	9
Avg. lead time per story (d)	15	20	18	17.6
Production deployments	8	13	11	32
<i>Throughput</i>				
Throughput (BCP/eng.-hour)	0.59	2.79	3.21	2.20
Hours per BCP	1.71	0.36	0.31	0.46

The delivery pattern across the three sprints showed a clear ramp-up profile. Sprint 1 closed with zero features completed despite seven user stories finished and 79 BCP delivered, reflecting the upfront cost of specification work, agent configuration, and the resolution of legacy integration contracts before any feature could be considered end-to-end ready. Sprints 2 and 3 then absorbed the bulk of delivery—two and three features respectively, with user-story output stabilizing at nine per sprint—and concentrated 91% of the project’s 890 BCP into the final two sprints. The five features and 25 user stories planned for six sprints under the four-engineer baseline were delivered in three, a 50% compression of the planned timeline.

Flow and throughput metrics tell the same story from a different angle. Average lead time per user story remained in a narrow 15–20 day band across all three sprints, indicating that the compression of headcount did not come at the cost of stories spending longer in the pipeline. Production deployment frequency was high and stable (8, 13, and 11 deployments per sprint, for a total of 32 over nine weeks), consistent with a workflow that pushed small increments through the guardrails continuously rather than batching releases. Throughput per engineer-hour evolved from 0.59 BCP/hour in Sprint 1 to 3.21 BCP/hour in Sprint 3—a 5.4 $\times$ increase—bringing the project average to 2.20 BCP/hour, or 0.46 hours per BCP. This trajectory is consistent with the learning-curve interpretation suggested by Sprint 1’s setup cost: once the agents were configured, the knowledge bases populated, and the legacy contracts mapped, the marginal cost of each subsequent unit of complexity dropped sharply. We return to the implications of this pattern, and its dependence on upfront specification work, in Section VI.

2) *Quality metrics*: Table III reports the quality metrics collected over the three sprints, grouped into test coverage, test execution, and compliance and defects.

TABLE III
QUALITY METRICS COLLECTED DURING THE STUDY

Metric	Sprint 1	Sprint 2	Sprint 3	Total
<i>Test coverage (average across sprints)</i>				
Backend coverage, JaCoCo (%)	95.7%	92.3%	90.4%	92.8%
Frontend coverage, Jest (%)	90.2%	90.1%	90.7%	90.3%
<i>Test execution</i>				
Integration tests defined	24	37	52	113
Integration tests passing	24	37	52	113
Integration test pass rate (%)	100%	100%	100%	100%
E2E tests (Cypress) executed	15	21	29	65
E2E tests passing	15	21	29	65
<i>Compliance and defects</i>				
WCAG 2.1 automated checks passed	0	0	2	2
WCAG 2.1 manual sign-offs	0	4	6	10
Defects found post-validation	0	1	0	1
Defects found post-release	0	0	0	0

The quality picture is consistent across the three categories reported in Table III. Backend coverage measured by JaCoCo averaged 92.8% across the project, starting at 95.7% in Sprint 1 and settling around 90% as the codebase grew; frontend coverage measured by Jest remained essentially flat at 90.3% throughout. The mild decline in backend coverage is the expected consequence of new code being added faster than tests in the later sprints, but the absolute level remained well above the institutional gate enforced by the pipeline. Both figures sit above the coverage thresholds typically required for production deployment in the institution’s regulated stack.

Test execution shows no degradation as scope grew. The number of integration tests defined nearly doubled between Sprint 1 and Sprint 3 (from 24 to 52, totaling 113), and the number of end-to-end Cypress tests grew in proportion (from 15 to 29, totaling 65), yet every test passed at every sprint boundary—an integration pass rate of 100% across 113 tests and an E2E pass rate of 100% across 65 tests. This pattern

is consistent with the SDD workflow described in Section III: with granular specifications driving both code generation and test generation, the expected behaviors, boundary conditions, and failure scenarios were encoded in tests before implementation began, leaving little room for drift between intent and outcome.

Compliance and defect signals are equally clean. Ten features received manual accessibility sign-offs against WCAG 2.1 AA across Sprints 2 and 3, complemented by two automated checks recorded in Sprint 3 as the accessibility tooling was added to the pipeline. Only one defect was found during post-validation testing, in Sprint 2, and no defects were recorded after release. Taken together with the throughput trajectory reported in the previous section, this suggests that the compression of headcount did not erode the quality bar: the squad delivered faster, deployed more frequently, and produced code that cleared the same automated and manual gates the four-engineer baseline would have had to clear.

B. RQ2: Under what conditions does the model succeed or break down?

During the nine-week period, three conditions consistently shaped the outcomes reported in Section V-A. [GP: *vale dizer como vocês identificaram essas três condições — emergiram dos logs de escalation? das retros? de revisão dos autores ao final? sem isso, parece “achismo dos autores”*]

Specification quality is the primary determinant. Thorough, unambiguous specifications consistently produced AI-generated code that required only minor adjustments. Vague or incomplete specifications produced unusable outputs regardless of tool, and this pattern held for both the supervised (Copilot) and the autonomous (Devin) workflows. In the brownfield context, undocumented legacy integration contracts were the dominant source of underspecification: when an existing behavioral contract was not made explicit in the specification, generated code violated it, and the rework cost exceeded what a more complete upfront specification would have required. [GP: *precisamos de números aqui: quantos eventos de escalation foram logados nos 3 sprints? quantos foram causados por specs incompletas vs. outras causas? sem isso, “dominant source” é afirmação não-suportada. se tiver 1-2 exemplos concretos (sem violar NDA), uma frase tipo “for example, in Sprint 2, a missing precondition on the LACUNA contract caused...” fortalece muito*]

The core/non-core partition is a repeatable heuristic. The dual-module strategy held across all five features: domain-intensive logic required continuous human judgment that autonomous agents could not reliably supply, while standardized infrastructure work could be fully delegated. The partition boundary was not always obvious upfront; it emerged from a simple observable signal—where Devin’s autonomous output required repeated human correction across iterations, the work was reclassified as core and moved to Copilot’s supervised loop. [GP: *quantas reclassificações aconteceram? em quais features? se foi “X módulos reclassificados nas primeiras 2*

sprints, 0 reclassificados na sprint 3”, isso vira evidência forte de estabilização. sem isso, “stabilized within the first two sprints” é assertion] The partition stabilized within the first two sprints and held without revision in Sprint 3, suggesting that the heuristic is learnable through early iteration rather than requiring upfront architectural foresight. [GP: poderia dar 1-2 exemplos do que ficou em core e do que ficou em non-core nesse projeto específico? ex: “validação de regras LACUNA → core; configuração de filas Kafka → non-core”. torna o achado replicável]

The model is a multiplier of existing expertise, not a substitute for it. The engineer had 8 years of professional experience and 4 years within the institution. [GP: vale caracterizar mais o perfil: T-shaped? full-stack? backend-heavy? a seção 6 menciona “T-shaped” como condição de transferibilidade — antecipar essa caracterização aqui amarra os dois argumentos] This expertise was what made AI output evaluable: without it, the quality gate that the removed team members would have provided would be absent. Becker et al. [3] point in the same direction, finding that experienced developers working on familiar codebases did not uniformly see the expected productivity gains from AI tools. Cui et al. [13], in a multi-company field experiment with 4,867 developers, observed a similar pattern: productivity gains from AI coding tools concentrated among junior developers, with senior developers familiar with their codebase seeing little or no measurable speed-up. The configuration we report inverts this relationship—the senior engineer’s experience is exactly what makes the gains possible—because the engineer is no longer writing code, but directing and evaluating the work of agents that do. [GP: essa virada é o achado teórico mais forte da seção; vale considerar dar mais peso, talvez 1-2 frases adicionais explicando o mecanismo: por que dirigir e avaliar exige expertise diferente de codificar? expertise de domínio vs. expertise de implementação?]

[GP: a tabela está OK como síntese, mas considere adicionar uma 5a linha sobre “team continuity / single point of failure” — a seção 6 levanta isso como limitação importante e fica estranho não aparecer aqui. ex: “Risk mitigation: documentation-as-artifact (SDD); single engineer with full mental model”]

Automated guardrails absorb the review work removed with the team. Removing three team members removes three review loops: peer code review, accessibility checks, and security validation as social-process activities. In this configuration, those loops were absorbed by automated gates in the CI/CD pipeline—WCAG 2.1 AA checks, security scans, coverage thresholds, and integration test execution—enforced before any artifact could progress to homologation. The guardrails were the mechanism that allowed the human quality gate to scale from one engineer to the work of four. Teams attempting this configuration without equivalent automated enforcement should expect quality outcomes closer to those of an unsupervised solo developer than to those reported here.

VI. LESSONS LEARNED

Beyond the headline metrics reported in Section V, the experiment surfaced a set of practical lessons that we believe are more useful to other enterprise teams than the delivery numbers themselves. We organize them into what worked, what did not, and what the conditions for transferability appear to be.

A. What Worked

The most consistent gain came from collapsing the outer loop of inter-discipline coordination. In a conventional squad, work crossing the boundaries of product analysis, architecture, security, and quality engineering accumulates wait time at each handoff: backlog refinement meetings, architecture reviews, security validations, and QA sign-offs each impose their own scheduling and context-switching overhead. When these disciplines were embodied as AI agents under the direction of a single engineer, the round-trip cost of cross-disciplinary questions dropped from days to minutes. Activities that previously required coordinating multiple calendars were resolved within a single working session. The compression of time-to-market reported in Section V is, in large part, a consequence of this collapsed outer loop rather than of faster individual coding.

A second observation is that AI agents proved effective at filling discipline-specific gaps in the engineer’s own profile. Areas where the staff engineer had less depth—accessibility evaluation, certain UI design decisions, infrastructure boilerplate—were handled by agents configured with the relevant procedural knowledge. In a traditional squad, those gaps would have been filled either by hiring or training a specialist, or by absorbing the cost of asynchronous consultation with experts in adjacent teams. The agents did not replace deep specialist judgment, but they did raise the floor of what a single engineer could competently deliver across disciplines.

B. What did not work

The most important counter-finding concerns the profile of the human in the loop. The model assumes a generalist with enough breadth to direct, evaluate, and correct AI output across multiple disciplines. The pool of engineers who fit this profile is smaller than the general population of engineers, both in our internal staffing patterns at Itaú and in the wider literature. Delicado et al. [14], in a qualitative study of the Spanish aerospace industry, document that deeply specialized engineers struggle to cross disciplinary boundaries, and propose the T-shaped competency model—depth in one area combined with breadth across adjacent disciplines—as a remedy. Empirical evidence on AI-augmented development reinforces the relevance of breadth: Cui et al. [13], in a multi-company randomized trial covering 4,867 professional developers, found that productivity gains from AI coding tools concentrated among junior and less-experienced developers, while senior developers already familiar with the codebase and stack saw little or no measurable speed-up. This pattern is consistent with Becker et al. [3] and suggests that AI augmentation rewards

TABLE IV
ENABLING AND LIMITING CONDITIONS FOR THE ONE-PERSON SQUAD MODEL

Dimension	Enabling condition	Limiting condition
Specification	Thorough, unambiguous; includes legacy contracts	Vague; undocumented integration behavior
Task type	Standardized, pattern-driven (infra, boilerplate)	Domain-intensive, semantically rich business logic
Engineer experience	Deep institutional and domain knowledge	Limited familiarity with codebase or domain
System context	Brownfield with documented constraints	Undocumented legacy; high implicit knowledge

adaptability across domains rather than depth within a single one. Scaling the one-person squad model across the institution would therefore require deliberate investment in growing T-shaped or generalist profiles, not simply granting access to AI tooling.

The second limitation is structural rather than skills-related: the one-person squad introduces a single point of failure. With one person carrying the full mental model of the product, an unplanned absence, a reassignment, or a departure leaves the project without continuity. The risk is not hypothetical; it is the natural consequence of compressing four people’s tacit knowledge into one head. Two practical mitigations emerged from our experience.

First, documentation has to be treated as a first-class artifact from day one, not an afterthought near release. The same Spec-Driven Development practices that enabled high-quality AI code generation also produced specifications, decision records, and agent configurations detailed enough that a different engineer—or even a different set of agents—could pick up the project mid-stream. SDD is not only a code-quality lever; it is also a continuity lever.

Second, we believe a more durable configuration is a two-person technical pair plus a fractional product strategist, rather than a literal one-person squad. The two engineers share the technical mental model and review each other’s direction of the AI agents, removing the single-point-of-failure problem while still capturing most of the compression gains. The product strategist is needed primarily during the early specification phase, where intent and prioritization decisions are made; once the specification is solid, this role becomes consultative rather than embedded. A floating product profile serving several small technical pairs is, in our reading, a more realistic operating model than the one-person extreme. We present this configuration as a hypothesis derived from the experiment rather than as a result we tested directly; controlled comparison between one-person, two-person, and three-person AI-augmented configurations is left for future work.

C. When more people add less value

We approach this point with caution because team-size questions carry real consequences for individual engineers. The boundary we observed is narrow: additional people add less marginal value when the work is well-specified, follows established institutional patterns, and concerns a domain the directing engineer already understands deeply. Brownfield projects within a familiar product area, where architecture and standards are settled and the unknowns are primarily about

implementation rather than design, fit this profile. Under these conditions, an additional engineer often spends more time being onboarded and coordinated than producing differentiated output, and the AI agents already cover the breadth that a junior teammate would have provided.

Conversely, additional people remain clearly valuable when the work involves genuine product uncertainty (where multiple competing visions need to be debated), unfamiliar domains (where the directing engineer would themselves be guessing), high-blast-radius systems (where independent review is a regulatory or risk-management requirement), or sustained operation over long horizons (where rotation, knowledge sharing, and on-call coverage matter). The one-person model is not a general substitute for team-based work; it is a configuration suited to a specific class of projects.

D. Comparison against the historical baseline

[GP: pode revisitar aqui?]

Two complementary readings are worth surfacing. Against the same squad’s historical throughput on prior projects of comparable scope in the same product domain, effort per BCP fell from 8.93 hours to 4.35 hours—a 51% reduction. Because the baseline and the experimental condition share the same product domain, codebase, and complexity-scoring instrument, this comparison controls for team familiarity with the system, but not for residual differences in feature mix across project cycles. Cost moved in the same direction: direct staffing fell from R\$492,000 to R\$60,000 (−88%), and an above-85% reduction holds even after internalizing the AI tooling subscriptions (R\$5,000–R\$7,000 estimated). Table V consolidates this view.

TABLE V
ONE-PERSON SQUAD VS. HISTORICAL BASELINE: DELIVERY AND COST

Metric	Baseline (4 engineers)	One-person squad
Team size	4 engineers	1 staff engineer
Delivery timeline	6 sprints (~18 wks)	3 sprints (~9 wks)
Effort per BCP	8.93 hours	4.35 hours (−51%)
Direct staffing cost	R\$492,000	R\$60,000 (−88%)
AI tooling cost	—	R\$5,000–R\$7,000
Adjusted reduction	—	>85%

In our context, the one-person squad matched the delivery and quality bar of a four-person team on the same complexity scale, at a fraction of the direct staffing cost, without eroding the automated and manual quality gates the four-engineer baseline would have had to clear. The cost covers direct staffing

and approximate tooling; a full Total Cost of Ownership would additionally account for ramp-up, infrastructure, and the overhead of configuring agents and curating their knowledge bases. Read with this caveat in mind: the result is best understood as a boundary test of what AI-augmented compression can achieve, not as a universally replicable operating model.

[GP: *parei aqui*]

E. Transferability to other Itaú projects

For these lessons to translate beyond the single project reported here, three conditions appear to matter.

The first is the existence of a settled architectural baseline. Projects within product areas that already have well-documented patterns, established CI/CD pipelines, and codified non-functional requirements (security, accessibility, observability) gain the most, because the AI agents can be configured against stable reference points. Projects that are still negotiating their architectural foundations are less suited to compression and benefit more from conventional team structures.

The second is the availability of generalist profiles, or a deliberate program to develop them. In our experience, the directing role is harder to staff than the AI tooling is to acquire. Identifying engineers with breadth across product reasoning, architecture, and quality, and giving them a path to develop the additional disciplines they lack, is a precondition rather than a side effect of adoption.

The third is institutional commitment to documentation and skill reuse as governance practices. The compression gains observed here depended on artifacts—specifications, agent configurations, accessibility checklists—that other teams could in principle reuse. Without a centralized governance layer, each team would re-derive these artifacts independently, and the marginal cost of adoption would absorb the marginal gain.

We do not claim these conditions are exhaustive, and we expect future projects within the institution to surface adjustments. What we observed in this case is that the productive unit of analysis is not “AI tools applied to a team” but “a workflow redesigned around how cognitive effort is distributed,” and that workflows transfer only when the supporting artifacts and profiles transfer with them.

VII. RELATED WORK

The work reported in this paper sits at the intersection of three lines of recent research: empirical studies on AI coding assistants and developer productivity, multi-agent systems for software engineering, and team composition under cognitive constraints. We review each in turn and locate our contribution against them.

A. AI Coding Assistants and Developer Productivity

Empirical evidence on the productivity effects of AI coding assistants has accumulated rapidly since 2022, and the picture it draws is more nuanced than early reports suggested. Peng et al. [2] ran a controlled experiment in which developers given access to GitHub Copilot completed a JavaScript task

55.8% faster than a control group, with larger gains for less experienced participants. Ziegler et al. [15], working with telemetry from real GitHub Copilot users at GitHub, found that perceived productivity correlates most strongly with the rate at which suggestions are accepted, and that this rate is itself a function of task type and developer experience. Cui et al. [13] extended this evidence to enterprise settings through a multi-company randomized trial covering 4,867 developers at Microsoft, Accenture, and a Fortune 100 firm: the average productivity gain was 26%, but it was concentrated in junior developers, with senior developers familiar with their codebase showing little or no measurable speed-up. Becker et al. [3], in a randomized study with experienced open-source maintainers, observed that AI tools *slowed* those developers down by 19% on issues in their own repositories, despite the developers themselves reporting a perceived speed-up. Sami et al. [16], in a systematic literature review of 37 peer-reviewed studies, synthesize this divergence and conclude that productivity effects are strongly mediated by task type, developer experience, and codebase familiarity.

Beyond aggregate metrics, qualitative work has examined how developers actually use these tools. Barke and colleagues [17], through a grounded-theory study of 20 programmers, identify two distinct interaction modes—*acceleration*, when the developer knows what to do and uses the assistant to get there faster, and *exploration*, when the developer is unsure and uses the assistant to survey options. The acceleration mode depends on the developer being able to decompose the task into well-understood microtasks before invoking the assistant, which connects directly to the role of upfront specification observed in our study. Liang et al. [18] surveyed the literature on large language models for software engineering and found that adoption is driven less by raw code generation and more by reduction in time spent searching for patterns and writing boilerplate—reinforcing the view that AI gains depend on what task is being delegated.

Two gaps in this body of work motivate our study. First, almost all of the cited studies examine individual developers using a single assistant; few examine teams of agents directed by a single human. Second, almost none examine brownfield enterprise contexts with regulatory constraints; the experimental tasks tend to be small, self-contained, or open-source. The case reported here adds a regulated brownfield data point, with multiple agents under unified human direction, to a literature still dominated by individual-developer greenfield experiments.

B. Multi-Agent Systems for Software Engineering

The shift from AI-as-assistant to AI-as-agent has produced a distinct research thread. He, Treude, and Lo [4], in an ACM TOSEM survey of LLM-based multi-agent systems for software engineering, map applications across the full software development lifecycle—requirements, design, implementation, review, testing—and characterize multi-agent systems as a way to manage real-world project complexity through specialization and coordination among agents. The configuration

we report instantiates this paradigm: four agents assigned to product analysis, specification, core development, and non-core development, coordinated by a single human engineer.

Most of the multi-agent SE literature, including the He, Treude, and Lo survey, focuses on benchmark performance—resolution rates on SWE-bench and similar suites—rather than on team-organizational outcomes. The question of what happens to a development team when its functional roles are absorbed by agents has not, to our knowledge, been addressed empirically in peer-reviewed venues. Our paper engages this question directly, treating the unit of analysis as the squad rather than the agent.

C. Team Composition, Cognitive Load, and Developer Experience

The team-size question is older than AI, but recent work has refined it for modern software engineering contexts. Fagerholm et al. [19], in an ACM Computing Surveys taxonomy covering half a century of research on cognition in software engineering, document how the field has progressively shifted from studying individual cognitive tasks (code reading, debugging) to studying cognitive demands across roles and team configurations. Gonçalves et al. [20], in a systematic mapping study, catalog the empirical methods used to measure developer cognitive load and identify role-switching and context-switching as recurring sources of extraneous load. Sweller’s cognitive load framework [21] provides the underlying theoretical vocabulary: intrinsic load is inherent to the task, extraneous load is imposed by how the task is presented, and germane load is the productive effort of schema construction. Reducing extraneous load—which is what role-switching and handoff coordination impose—is therefore expected to improve sustained productivity, a prediction the developer experience literature confirms [22], [23].

Closer to our setting, Delicado et al. [14], through a qualitative study in the Spanish aerospace industry, document the practical limits of deep specialization in collaborative engineering work and propose the T-shaped competency model—depth in one domain combined with breadth across adjacent ones—as a remedy. Their findings inform our reading of which engineer profile is feasible in the directing role of a one-person AI-augmented squad. The configuration we report compresses four cross-functional roles into a single human plus four agents, and the cognitive load literature above predicts that this compression succeeds only when the human’s role-switching cost is offset by the agents absorbing routine work—a prediction our results are consistent with.

What this body of work does not yet address is the specific configuration we report: a single experienced engineer in a brownfield regulated setting, with multi-agent support, evaluated against the same team’s historical baseline rather than against a planning estimate or a benchmark. The case study reported here is, to our knowledge, the first peer-reviewed account of that configuration.

VIII. LIMITATIONS AND THREATS TO VALIDITY

[GP: limitacao do participante-observador] This study has several limitations that should inform how its findings are interpreted. First, and most fundamentally, the single-case design bounds what can be claimed. The study reflects one project within one financial institution, and findings may not generalize to other domains, regulatory environments, or technology stacks. Within that constraint, the one-person squad model was tested at its extreme boundary—a single engineer—and intermediate configurations such as two- or three-person AI-augmented squads were not investigated; those configurations may offer more practical and generalizable operating models for most enterprise contexts.

Second, the practitioner-researcher duality introduces potential confirmation bias, since one of the authors served as both the primary subject and the principal observer. Outcome metrics were defined prior to project execution and assessed against pre-established thresholds rather than post-hoc judgments to partially mitigate this, but the absence of an independent observer throughout the nine-week period remains a structural limitation of the design. Third, and closely related, the productivity baseline comes from the same squad’s historical velocity on prior projects in the same domain rather than from a parallel control group; while stronger than a planning estimate, this comparison cannot fully control for differences in feature complexity, technical debt, and external dependencies across project cycles.

Fourth, the indicators used to characterize the brownfield context—legacy integration points, compliance constraints, human escalation events—were not collected with a standardized instrument and were partially reconstructed from retrospective notes and repository history, which limits their reproducibility and makes cross-case comparisons difficult. Fifth, cost figures reported cover direct staffing and approximate tooling costs; a full Total Cost of Ownership analysis would additionally account for ramp-up time, infrastructure, and organizational overhead for agent configuration, which are approximated but not audited in this study. Sixth, engineer experience operates simultaneously as the mechanism that makes the model work and as a confound that limits generalizability: the results reflect what a staff engineer with 8 years of professional experience and 4 years within the institution can achieve when directing and evaluating AI output, and the same experiment with a less experienced engineer would likely produce different outcomes.

Finally, several dimensions relevant to enterprise AI adoption fall outside the scope of this study and merit attention in future work: the security implications of AI-generated code in regulated environments, intellectual property and licensing considerations for AI-generated artifacts, the long-term effect of AI-augmented workflows on developer skill development, the sustainability and well-being implications of one-person operating models, the labor market effects of team size reduction, and the risk of LLM hallucinations in production code.

IX. CONCLUSION

This paper reported a single-case study of a one-person AI-augmented squad delivering a brownfield product initiative at a large Brazilian financial institution under regulatory constraints. An engineer supported by four AI agents operating under a Spec-Driven Development workflow delivered a project originally scoped for a four-person squad in half the planned sprints, achieving 90% first-review acceptance of AI-generated code, full integration test coverage, WCAG 2.1 AA compliance, and an above-85% reduction in direct staffing cost. The results indicate that one-person AI-augmented delivery is feasible at this complexity scale, but the key constraint was not AI capability—it was the directing engineer’s institutional knowledge and the quality of specifications produced upstream. The model functions as a multiplier of existing expertise, not a substitute for it. Structural limitations—single case, practitioner-researcher duality, and a same-team historical baseline—bound what can be claimed, and the one-person configuration is best read as a boundary test rather than a target operating model. Useful next steps include controlled comparisons across one-, two-, and three-person AI-augmented configurations, replications in greenfield and less regulated contexts, and longitudinal studies of how engineer experience interacts with AI capability over time.

ARTIFACT AVAILABILITY

The source code, internal specifications, agent configurations, and quantitative measurements produced during the case study are proprietary assets of the financial institution and are subject to banking secrecy, confidentiality agreements, and internal information security policies. As a result, the primary artifacts of the study (source repositories, skill definitions, and build/test pipelines) cannot be made publicly available. The methodological protocol, AI agent role descriptions, and the anonymized measurement scheme used to report the results are available from the corresponding author upon reasonable request and subject to institutional approval.

REFERENCES

- [1] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Reading, MA: Addison-Wesley, 1975.
- [2] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirel, “The impact of AI on developer productivity: Evidence from GitHub Copilot,” *arXiv preprint arXiv:2302.06590*, Feb. 2023. [Online]. Available: <https://arxiv.org/abs/2302.06590>
- [3] J. Becker, N. Rush, E. Barnes, and D. Rein, “Measuring the impact of early-2025 AI on experienced open-source developer productivity,” *arXiv preprint arXiv:2507.09089*, Jul. 2025. [Online]. Available: <https://arxiv.org/abs/2507.09089>
- [4] J. He, C. Treude, and D. Lo, “LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, May 2025.
- [5] E. Gil, “The collapse of engineering team size,” *Elad Blog*, 2024. [Online]. Available: <https://blog.eladgil.com/>
- [6] McKinsey and Company, “The state of AI in 2025: Agents, innovation, and transformation,” *McKinsey Global Survey*, Nov. 2025. [Online]. Available: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>
- [7] G. Rosa, D. Moreno-Lumbreras, G. Robles, and J. M. González-Barahona, “Understanding specification-driven code generation with LLMs: An empirical study design,” 2026, to appear, *SANER 2026*.
- [8] G. Pinto, C. R. B. de Souza, J. B. Neto, A. de Souza, T. Gotto, and E. Monteiro, “Lessons from building stackspot AI: A contextualized AI coding assistant,” in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 2024, pp. 408–417. [Online]. Available: <https://doi.org/10.1145/3639477.3639751>
- [9] R. K. Yin, *Case Study Research and Applications: Design and Methods*, 6th ed. Thousand Oaks, CA, USA: SAGE Publications, 2018.
- [10] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [11] CI&T, “Business complexity points,” <https://ciandt.com/us/en-us/complexitypoints>, 2015, accessed: 2026-04-25.
- [12] W3C, “Web content accessibility guidelines (WCAG) 2.1,” W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [13] Z. K. Cui, M. Demirel, S. Jaffe, L. Musolff, S. Peng, and T. Salz, “The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers,” *SSRN Electronic Journal*, 2024.
- [14] B. A. Delicado, A. Salado, and R. Mompó, “Conceptualization of a T-shaped engineering competency model in collaborative organizational settings: Problem and status in the Spanish aircraft industry,” *Systems Engineering*, vol. 21, no. 6, pp. 534–554, 2018.
- [15] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, “Measuring GitHub Copilot’s impact on productivity,” *Communications of the ACM*, vol. 67, no. 3, pp. 54–63, 2024.
- [16] A. Mohamed, M. Assi, and M. Guizani, “The impact of LLM-assistants on software developer productivity: A systematic review and mapping study,” *arXiv preprint arXiv:2507.03156*, 2025.
- [17] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How programmers interact with code-generating models,” *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 7, no. 1, pp. 85–111, 2023.
- [18] J. T. Liang, C. Yang, and B. A. Myers, “A large-scale survey on the usability of AI programming assistants: Successes and challenges,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.
- [19] F. Fagerholm, M. Felderer, D. Fucci, M. Unterkalmsteiner, B. Marculescu, M. Martini, L. G. W. Tengberg, R. Feldt, B. Lehtelä, B. Nagyvárad, and J. Khattak, “Cognition in software engineering: A taxonomy and survey of a half-century of research,” *ACM Computing Surveys*, vol. 54, no. 11s, pp. 1–36, 2022.
- [20] L. Gonçalves, K. Farias, L. Kupssinskü, and M. Segalotto, “Measuring the cognitive load of software developers: An extended systematic mapping study,” *Information and Software Technology*, vol. 136, p. 106573, 2021.
- [21] J. Sweller, P. Ayres, and S. Kalyuga, *Cognitive Load Theory*. New York, NY, USA: Springer, 2011.
- [22] A. Noda, M.-A. Storey, N. Forsgren, and M. Greiler, “DevEx: What actually drives productivity,” *ACM Queue*, vol. 21, no. 2, pp. 35–53, 2023.
- [23] A. Razzaq, J. Buckley, Q. Lai, T. Yu, and G. Botterweck, “A systematic literature review on the influence of enhanced developer experience on developers’ productivity: Factors, practices, and recommendations,” *ACM Computing Surveys*, vol. 57, no. 1, pp. 1–46, 2024.